Attention-Aware Study Assistant

An agentic, attention-aware learning system with microservices + GenAI

GenAI System Building Challenge | Lab Hackathon

Team Project

May 2026

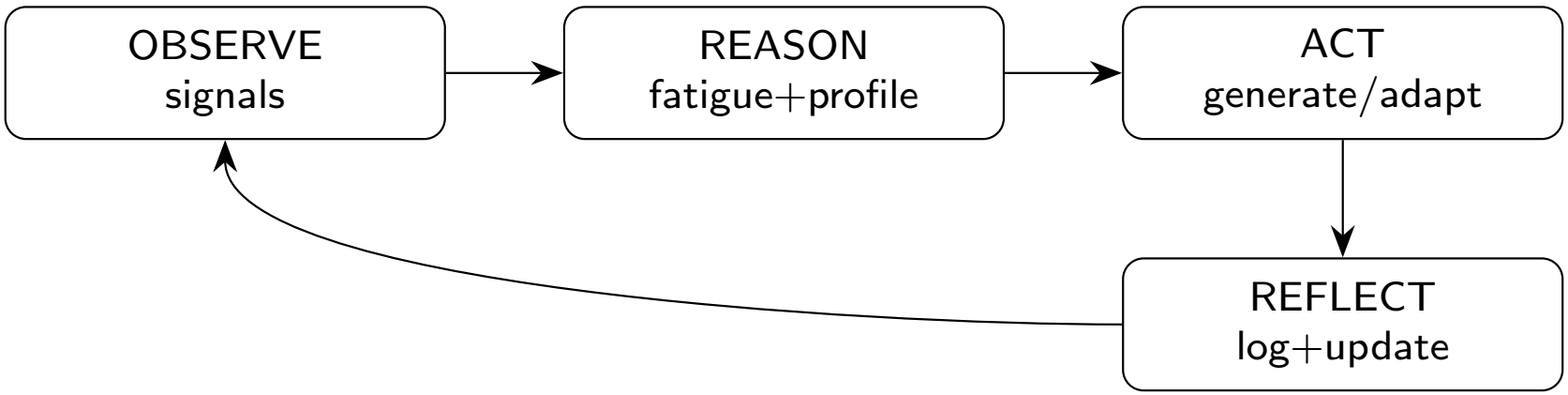
Agenda

- Problem & Idea
- System Overview
- Microservices
- GenAI & Pipelines
- Fatigue Detection
- Frontend & UX
- Quality & Deployment
- Demo & Takeaways

Core Idea: Attention-Aware Adaptation

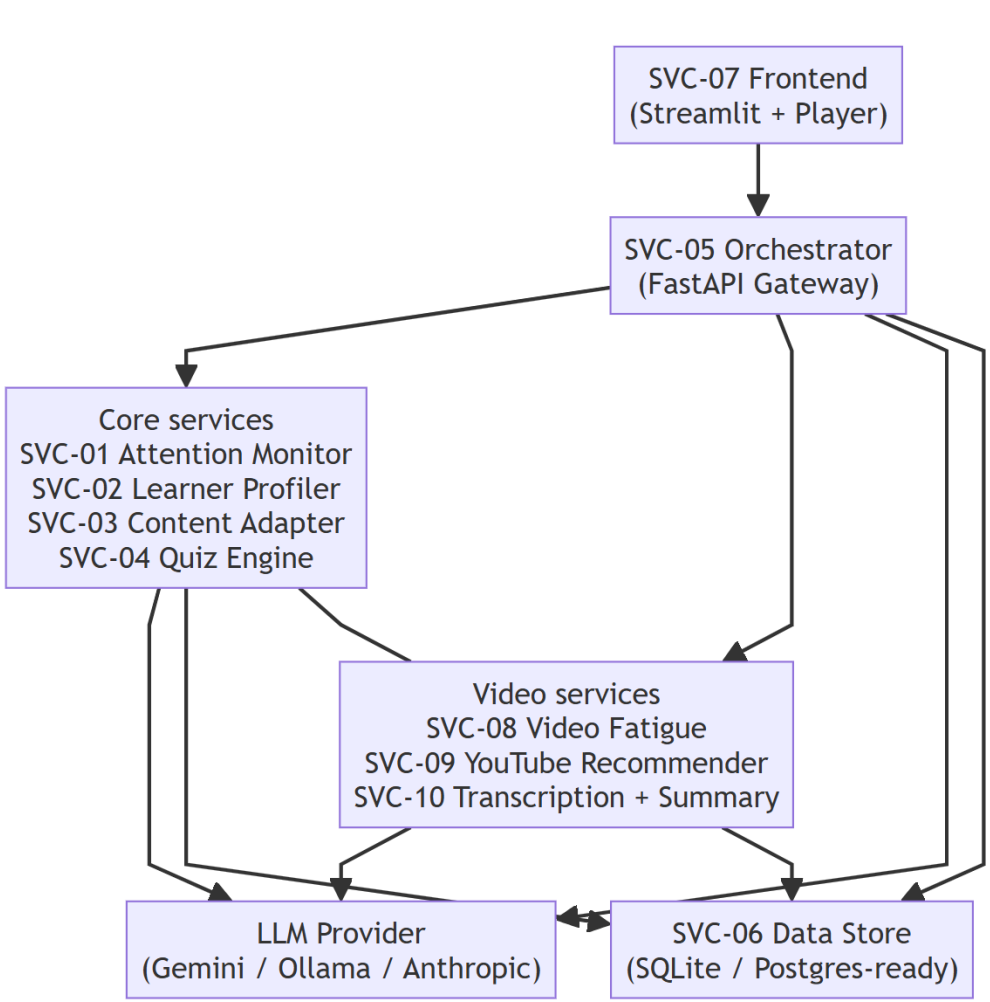
- Learners fatigue during long explanations (text or video) and comprehension drops.
- Most tutoring systems deliver static output regardless of attention state.
- We deliver **real-time adaptation** of content *and* learning strategy.
- Observe behavioral signals (keystrokes, response time, video interactions).
- Infer fatigue label: `FRESH` $\rightarrow$ `MODERATE` $\rightarrow$ `TIRED` $\rightarrow$ `EXHAUSTED`.
- Select best teaching mode (detailed / concise / analogy / quiz).
- Close the loop with persistence + profile updates.

Agentic Loop (ReAct)

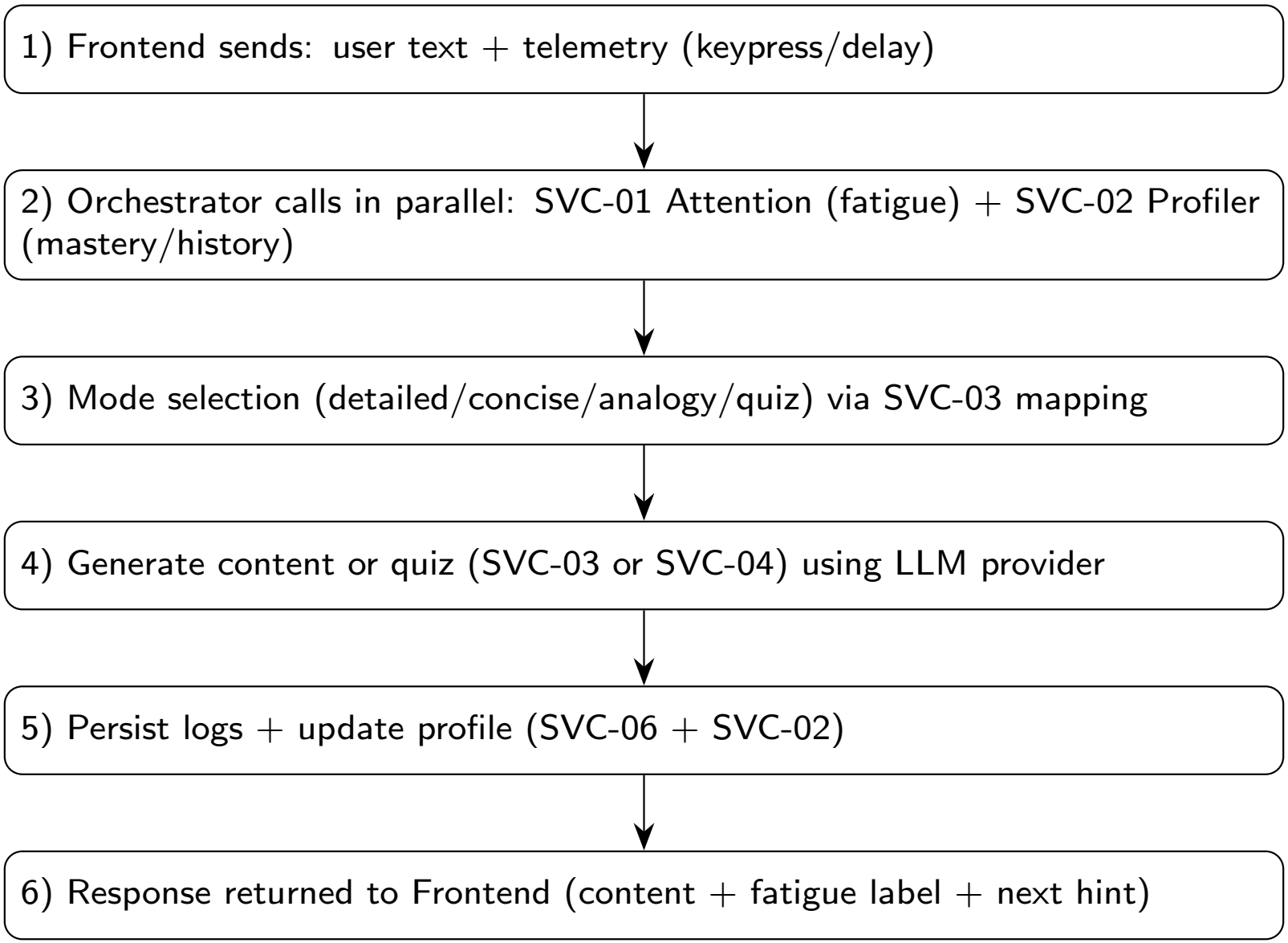


The Orchestrator runs this loop on every interaction.

High-Level Architecture (10 Microservices)



What Happens on `/interact`? (Flowchart)



Microservices at a Glance

ID	Port	Responsibility
SVC-01	8001	Attention monitor: fatigue from keystrokes, delay, quiz trend
SVC-02	8002	Learner profiler: mastery, difficulty, preferred mode, avg fatigue
SVC-03	8003	Content adapter: fatigue→mode mapping + prompt templating (Jinja2)
SVC-04	8004	Quiz engine: generate + evaluate (LLM-as-a-judge)
SVC-05	8000	Orchestrator: ReAct loop hub, API gateway routing
SVC-06	8005	Data store: sessions, interactions, profiles, quiz, video events
SVC-07	8501	Streamlit frontend: learning + quiz UI, gauges, history
SVC-08	8006	Video fatigue: score from player events + exponential decay
SVC-09	8007	YouTube recommender: topic → ranked videos (API/scrape)
SVC-10	8008	Transcription summary: transcript fetch (3-tier) + structured summary

Orchestrator: Hub-and-Spoke

- All inter-service communication flows through the Orchestrator.
- Enables independent deployability and clean separation of concerns.
- Key endpoints:
  - `POST /interact` (text learning + quiz flow)
  - `POST /video-interact` (video event telemetry)
  - `GET /health` (service health checks)

Data Persistence (SVC-06)

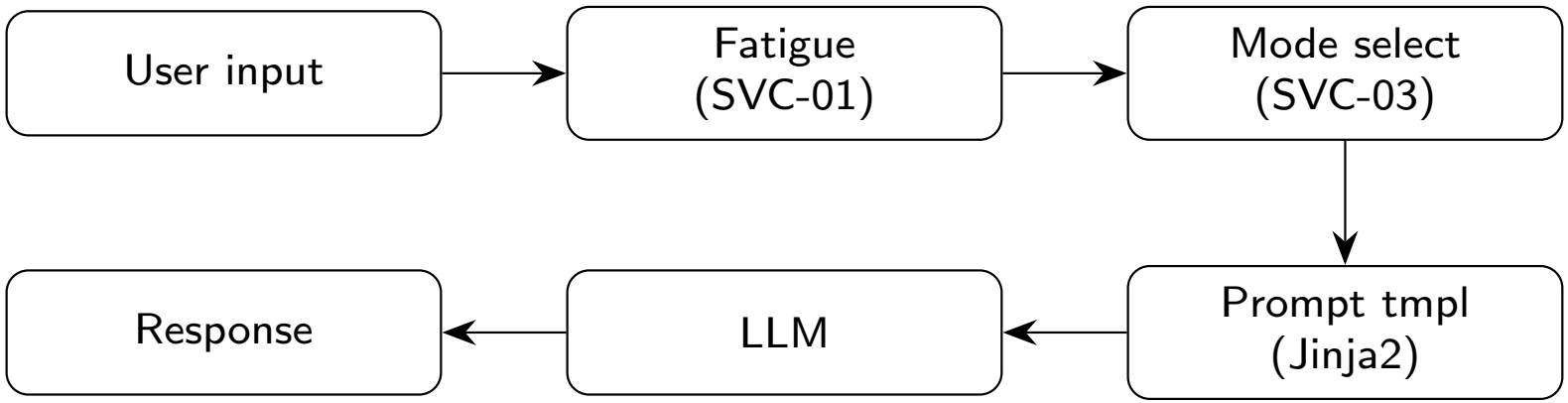
- SQLAlchemy ORM over SQLite (dev), designed for Postgres (prod).
- Core tables (audit trail + personalization):

Table	**Purpose**
`sessions`	session metadata, topic, start/end
`interactions`	every prompt/response with mode + fatigue
`learner_profiles`	mastery, difficulty, preferences, avg fatigue
`quiz_records`	questions, answers, score, feedback
`video_events`	player events + computed fatigue timeline

LLM Provider Abstraction

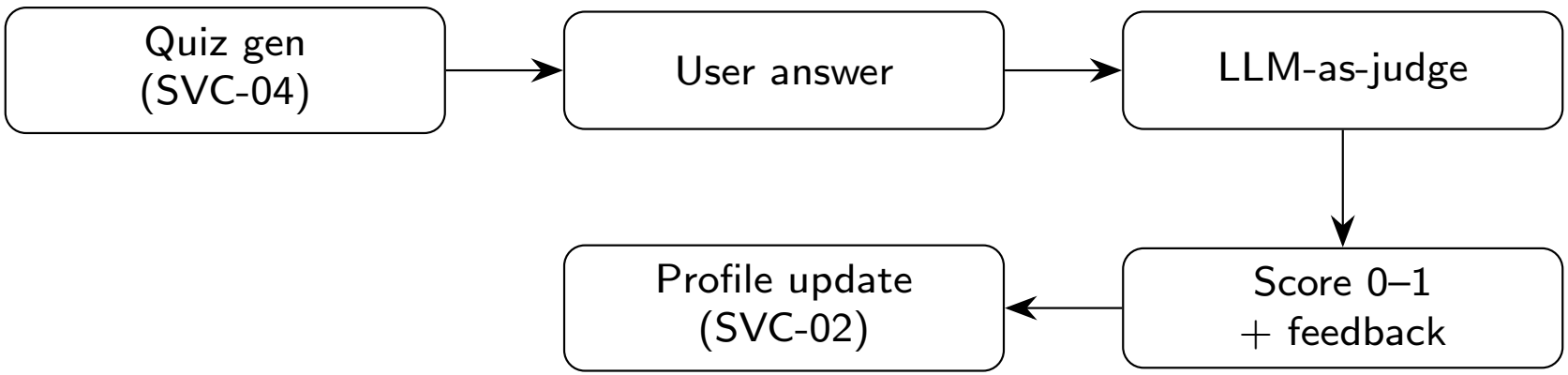
- Single interface: `generate(system_prompt, user_message)`.
- Provider priority (auto-detect):
  - Gemini API key $\rightarrow$ primary (fast, high-quality)
  - Ollama local $\rightarrow$ offline mode
  - Anthropic $\rightarrow$ optional fallback
- Keeps microservices provider-agnostic.

Pipeline: Content Generation

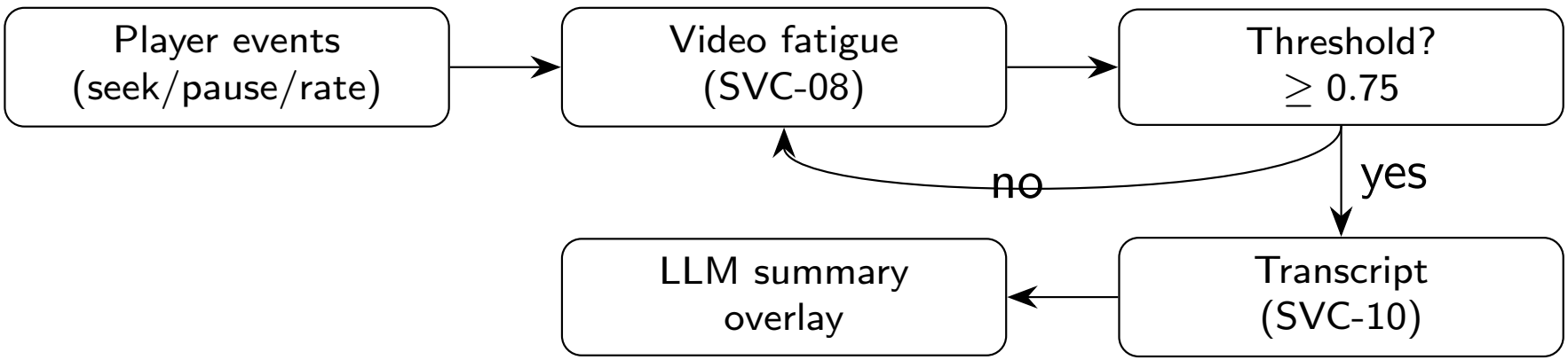


Everything is logged to SVC-06 for analytics and replay.

Pipeline: Quiz Generation & Evaluation



Pipeline: Video Intelligence + Summary Fallback



Fatigue Signals (Text vs Video)

Signal	**Text / Quiz Mode**	**Learning / Video Mode**
Primary telemetry	Keypress variance + response delay + quiz trend	Seek backward, pauses, speed changes, skips
Scoring	Weighted heuristic → label	Event impacts + exponential decay
Adaptation	Mode switch + quiz trigger	Suggest slowdown / show summary overlay

- Goal: keep cognitive load within a comfortable range.
- System reacts before the learner disengages.

Frontend: Learning Mode

- Embedded YouTube player + fatigue gauge (SVG/Plotly).
- Buttons: play/pause/seek/speed + one-click explanation.
- Local gauge blending (smooth UX) + server sync.
- When fatigue crosses threshold: summary overlay replaces remaining video.

Frontend: Quiz Mode (Telemetry + Feedback)

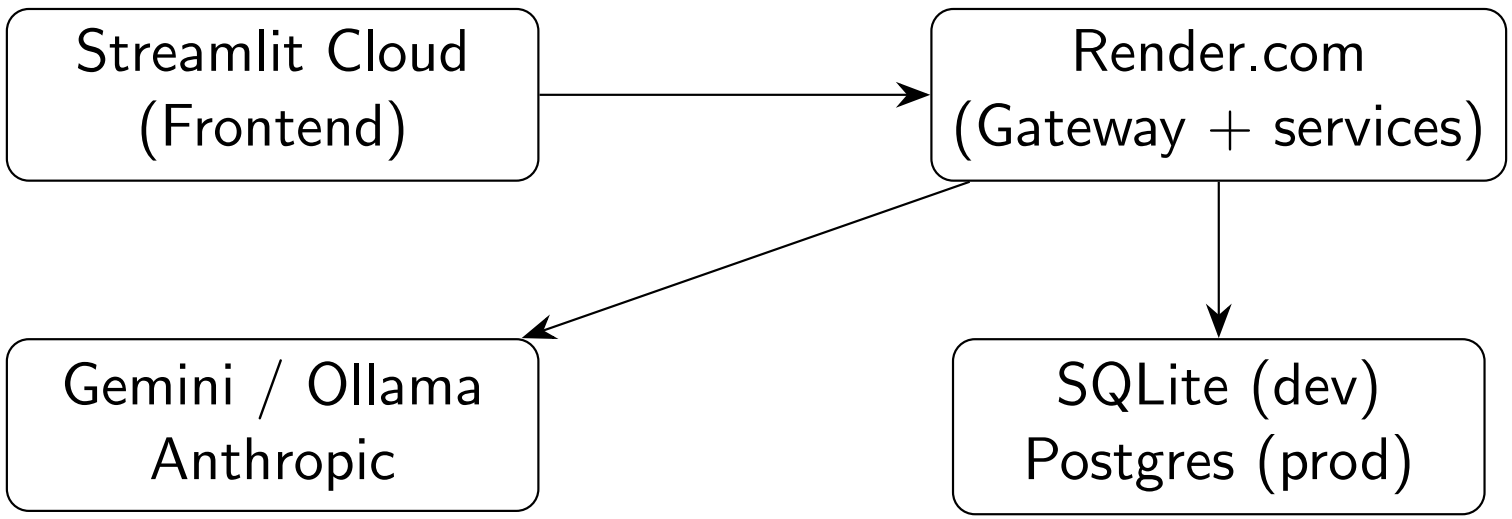
- Prompt-only interaction with live fatigue gauge in sidebar.
- Keypress telemetry captured in-browser and sent with each submission.
- Quiz evaluation returns: score, feedback, correct answer + explanation.

```
textarea.addEventListener('keydown', () => {
  timestamps.push(Date.now());
});
// intervals -> SVC-01 for fatigue inference
```

Testing Strategy

- 8 pytest modules, 30+ tests across key services.
- Focus: fatigue scoring correctness, mode mapping, datastore CRUD, pipelines.
- Orchestrator integration tests validate full ReAct loop.

Cloud-Native Deployment (Concept)



Demo Walkthrough (Suggested)

1. Enter a topic and request an explanation (FRESH $\rightarrow$ detailed).
2. Keep interacting until fatigue rises (mode becomes concise/analogy).
3. Trigger quiz and show evaluation + profile update.
4. Switch to video learning; demonstrate backward seeks raising fatigue.
5. Cross 75% threshold: show transcript summary overlay.

Conclusion

- A microservices + GenAI system that adapts in real-time to learner fatigue.
- ReAct orchestration keeps the architecture modular and testable.
- Dual modality: text/quiz adaptation + video intelligence and summary fallback.

Next steps (optional): calibration with real user studies, richer fatigue models, and tighter latency budgets.